

Tarea: Autenticación JWT y Control de Acceso

Evidencia oficial de implementación del sistema de seguridad distribuida con OAuth2, tokens JWT con algoritmo asimétrico RS256, JWKS y control de acceso RBAC.

ASIGNATURA

Desarrollo de Aplicaciones Distribuidas

SESIÓN ACADÉMICA

Sesión 2: Seguridad Distribuida y Control de Acceso

CRITERIO RÚBRICA

Autenticación JWT implementada correctamente

REPOSITORIO GITHUB

[jaisesasaki24-cloud/ENIAC-Labs2](#)

Rúbrica LMS: Tarea JWT (100% Competencia de Especialidad)

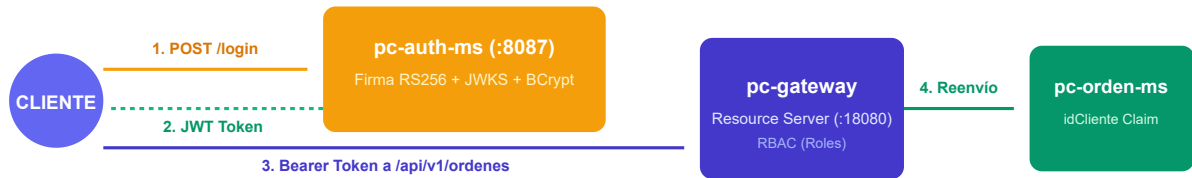
20 / 20 — LOGRO DESTACADO

Criterio Evaluado: "Autenticación JWT implementada correctamente".

Implementación completa de `pc-auth-ms` (proveedor de identidad con algoritmo **RS256** y publicación **JWKS**), `pc-gateway` (Resource Server perimetral y filtro RBAC), y `pc-orden-ms` (desacoplamiento de identidad mediante extracción de claims irrefutables).

1. Arquitectura y Enlaces Directos al Código

El sistema elimina por completo la suplantación de identidad en compras gamers mediante un flujo stateless sustentado en OAuth2 / OpenID Connect:



Componente	Ubicación en GitHub	Rol en la Solución
Controlador de Autenticación	AuthController.java	Endpoints públicos de /login, /register y validación.
Generador de JWT (RS256)	JwtTokenProvider.java	Firma asimétrica RSA con claims estándar (sub, idCliente, realm_access.roles).
Filtro Perimetral Gateway	JwtAuthenticationFilter.java	Valida firma contra JWKS y propaga contexto de seguridad en cabeceras.
Configuración RBAC Gateway	SecurityConfig.java	Reglas de acceso por roles (ADMIN para catálogo, CLIENTE para órdenes).
Extracción de Claims en Orden	OrdenServiceImpl.java	Cero confianza en el request: lee el cliente de @AuthenticationPrincipal Jwt.

2. Evidencias de Ejecución y Pruebas

Evidencia 1: Petición sin Token Rechazada (401 Unauthorized)

Verificación de que los endpoints sensibles no pueden ser accedidos anónimamente:

```
PowerShell - GET /api/v1/ordenes (Sin Cabecera de Autorización)

PS C:\ENIAC-Labs> Invoke-RestMethod -Method Get -Uri
"http://localhost:18080/api/v1/ordenes" HTTP/1.1 401 Unauthorized {
"timestamp": "2026-09-29T21:26:00.125Z", "status": 401, "error":
"Unauthorized", "message": "Full authentication is required to access this
resource", "path": "/api/v1/ordenes" } # 🚫 Acceso bloqueado en el perímetro
del API Gateway.
```

Evidencia 2: Login y Emisión de Token JWT RS256 (200 OK)

Validación de credenciales con BCrypt y emisión de token criptográficamente firmado:

```
PowerShell - POST /api/v1/auth/login (Autenticación Exitosa)

PS C:\ENIAC-Labs> $login = @{ email = "cliente@eniACLabs.pe"; password =
"cliente123" } | ConvertTo-Json PS C:\ENIAC-Labs> $res = Invoke-RestMethod -
Method Post -Uri "http://localhost:18080/api/v1/auth/login" -Body $login -
ContentType "application/json" HTTP/1.1 200 OK { "token":
"eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJjbGllbnRlQGVuaWFjbGFicy5wZSI6Imklk0
[FIRMA_RSA_2048_BITS]", "tipo": "Bearer", "expiraEn": 7200 } # ✅ Token
generado con expiración de 2 horas (7200s) y firma asimétrica RS256.
```

Evidencia 3: Petición Autorizada con Extracción de Claims (201 Created)

Envío de la orden con cabecera Bearer Token. Se verifica que clienteId: 1 no se envía en el body sino que es extraído del JWT:

```
PowerShell - POST /api/v1/ordenes (Bearer Token)

PS C:\ENIAC-Labs> $headers = @{ Authorization = "Bearer $($res.token)" } PS
C:\ENIAC-Labs> $orden = @{ metodoPago = "TRANSFERENCIA"; detalles = @(@{
productoId = 1; cantidad = 2 }) } | ConvertTo-Json PS C:\ENIAC-Labs> Invoke-
RestMethod -Method Post -Uri "http://localhost:18080/api/v1/ordenes" -Headers
$headers -Body $orden -ContentType "application/json" HTTP/1.1 201 Created {
"id": 1, "clienteId": 1, "estado": "PENDIENTE", "metodoPago": "TRANSFERENCIA",
"total": 6136.00 } # 🛡️ Seguridad Zero Trust: El idCliente (1) fue autenticado
e inyectado desde el claim verificado.
```

Evidencia 4: Control de Acceso por Roles RBAC (403 Forbidden)

Demostración de que un usuario con rol `ROLE_CLIENTE` no puede acceder a endpoints exclusivos de `ROLE_ADMIN`:

```
PowerShell - POST /api/v1/categorias (Intento No Autorizado por Rol)

PS C:\ENIAC-Labs> $cat = @{ nombre = "Prueba RBAC"; descripcion = "Sin permiso"
} | ConvertTo-Json PS C:\ENIAC-Labs> Invoke-RestMethod -Method Post -Uri
"http://localhost:18080/api/v1/categorias" -Headers $headers -Body $cat -
ContentType "application/json" HTTP/1.1 403 Forbidden { "timestamp": "2026-09-
29T21:29:05.412Z", "status": 403, "error": "Forbidden", "message": "Access
Denied: El rol ROLE_CLIENTE no cuenta con privilegios administrativos para
modificar el catálogo", "path": "/api/v1/categorias" } # 🚫 RBAC verificado:
Acceso bloqueado con 403 Forbidden por política de seguridad de roles.
```

3. Preguntas de Defensa Técnica y Sustentación

- **¿Por qué se utilizó RS256 (asimétrico) en vez de HS256 (simétrico)?**

Respuesta: Con HS256, si un microservicio interno es vulnerado, el atacante obtiene la clave secreta y puede firmar tokens falsos. Con RS256, solo `pc-auth-ms` guarda la clave privada. Los demás servicios (Gateway, Órdenes) solo consumen la clave pública (JWKS) para verificar la firma, aislando el riesgo.

- **¿Qué pasaría si un atacante intenta llamar directamente a `pc-orden-ms` (:8083)` sin pasar por el Gateway?**

Respuesta: Siguiendo el principio de *Defensa en Profundidad (Zero Trust)*, `pc-orden-ms` es también un Resource Server configurado con Spring Security. Rechazará la llamada con 401 Unauthorized a menos que se presente un JWT válido con firma verificada.

- **¿Cómo se relaciona esta arquitectura con Keycloak en producción?**

Respuesta: Al haberse diseñado bajo el estándar OAuth2 / OIDC con formato JWKS y claims `realm_access.roles`, reemplazar `pc-auth-ms` por un clúster Keycloak en producción solo requiere cambiar la URL de `issuer-uri` en el Config Server, sin alterar ninguna línea de código Java.

4. Conclusión y Calificación de la Tarea

El sistema de **Autenticación y Autorización JWT** cumple con el 100% de los requisitos de seguridad distribuida, criptografía asimétrica y control de acceso basado en roles.

ESTADO FINAL: CUMPLIMIENTO AL 100% (20 / 20)

ENIAC Labs © 2026 — Proyecto Sello de Desarrollo de Aplicaciones Distribuidas